

Computer Hardware

A Brief History of Concurrency: From a Button to a Scheduler

Ksawery Moźdzynski

Agenda For Today

1. Interrupts

The device tells the processor that something happened

2. The scheduler

One processor, several tasks

3. FreeRTOS

A real scheduler on the ESP32-C3

Today you will need

Wokwi



wokwi.com

your project Circuit from Lecture 5

Simulators

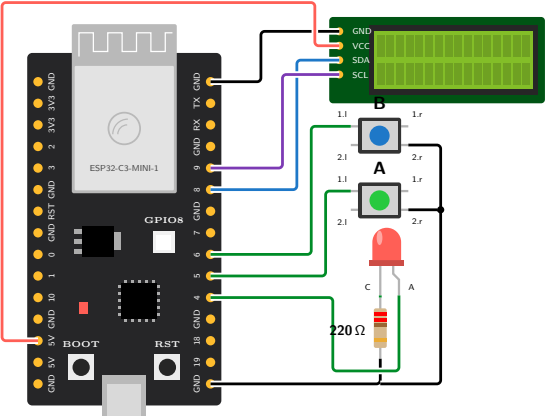


ksawerym.github.io/ComputerHardwareSimulators

No Circuit project from Lecture 5? Download it from GitHub:

github.com/KsaweryM/Computer-Hardware-Course/tree/main/Lecture_6_A_Brief_History_of_Concurrency/Circuit

From last time: our circuit

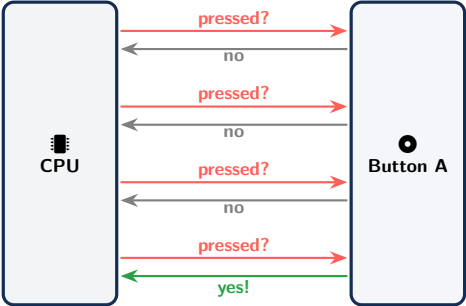


Part	Pin
LED	GPIO 4
Button A	GPIO 5
Button B	GPIO 6
LCD (SDA, SCL)	GPIO 8, 9

Buttons
 Pressed = 0, not pressed = 1.

From last time: what is polling?

Polling: the CPU reads the device again and again until something happens.



Polling is simple: one loop that keeps **asking** the button.

Polling is like checking your mailbox



You walk to the mailbox again and again. Almost every time it is **empty**.

The CPU does the same with the button: it walks to it and looks, again and again.

How often should the CPU check the button?



From time to time

Do some work, then check once, then work again.



All the time

Do nothing else: check, check, check.

We tried both last time. Let us look at each one.

Check from time to time: the code



```
int count = 0;

void loop() {
    delay(2000);           // "work": 2 s
    if (digitalRead(BUTTON_A) == LOW) {
        count++;          // check once
    }
    lcd.setCursor(0, 1);
    lcd.print(count);
}
```

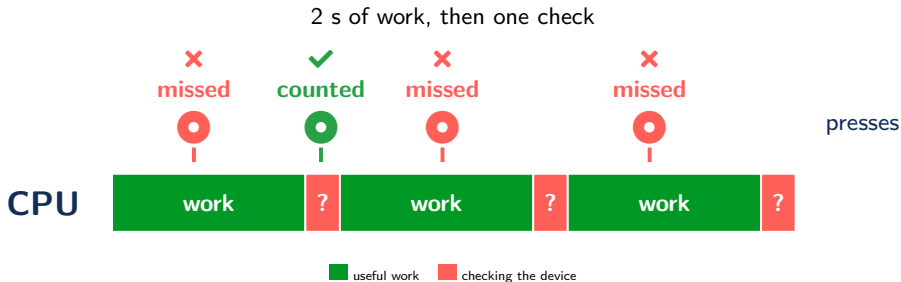
Work

delay(2000) stands for 2 s of other work.

One check

Then the program reads the button **once**, and goes back to work.

Check from time to time: what happens?



A press counts only if the button is down **exactly** when the CPU checks it (green).
Short presses during the work are **lost** (red).

Check from time to time: we miss presses



Presses are lost

Last time

We pressed button A 5 times, shortly. The LCD showed fewer than 5, often 0 or 1.

Why

A short press can come and go **between two checks**. The CPU never sees it.

Checking rarely leaves time for work, but the program **misses events**.

Check all the time: the code



```
int last = HIGH;    // the last state
long checks = 0, presses = 0;

void loop() {
  int now = digitalRead(BUTTON_A);
  checks++;
  if (last == HIGH && now == LOW) {
    presses++;      // a new press
  }
  last = now;
}
```

No work

loop() does nothing but read the button.

A new press

It counts a press when the button was HIGH last time and is LOW now.

Check all the time: what happens?



Every press is noticed, but almost every check finds **nothing**.

Check all the time: we waste the CPU



The CPU is wasted

Last time

In 10 s the CPU checked the button hundreds of thousands of times. More than **99.99%** of the checks found nothing.

Why

All the CPU time goes to asking. There is **no time left** for real work.

Checking all the time misses nothing, but it **wastes the CPU**.

Neither way is good



From time to time

time for work, but **presses are lost**



All the time

no press is lost, but **the CPU is wasted**

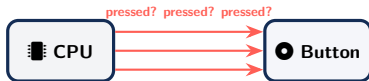
So what now?

How can the CPU do its work **and** never miss a press?

The problem: with polling the CPU has to **ask**. It cannot know when the button is pressed.

The solution: interrupts

Polling: the CPU asks



Interrupt: the button tells the CPU

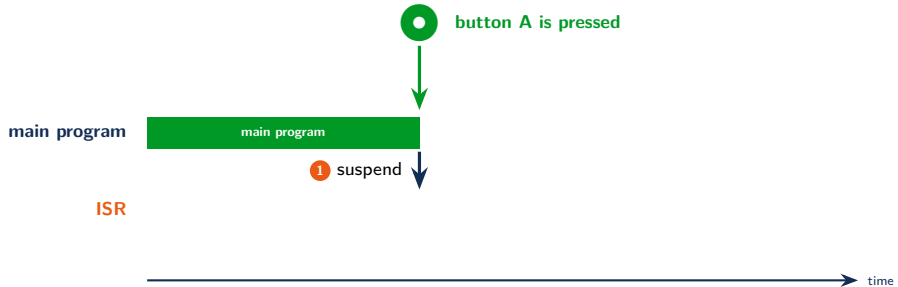


The solution is an **interrupt**: the device itself tells the processor that something happened. Until then, the CPU does its own work.

No missed presses, and no time wasted on asking. Let us see how it works.

An interrupt: 1. the signal

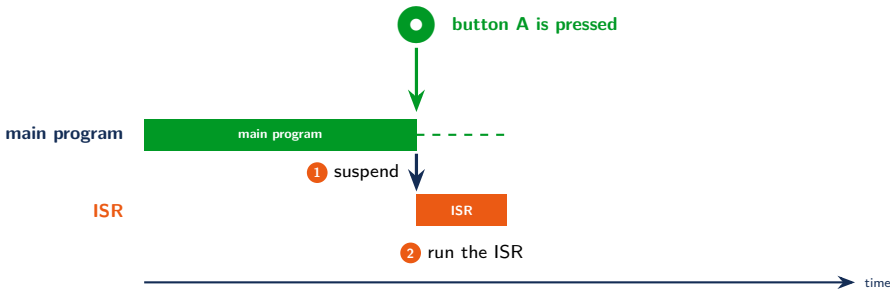
An **interrupt** is a signal from a device to the processor: *something happened, deal with it now.*



1 Button A is pressed. The processor **suspends** the main program at once.

An interrupt: 2. the ISR runs

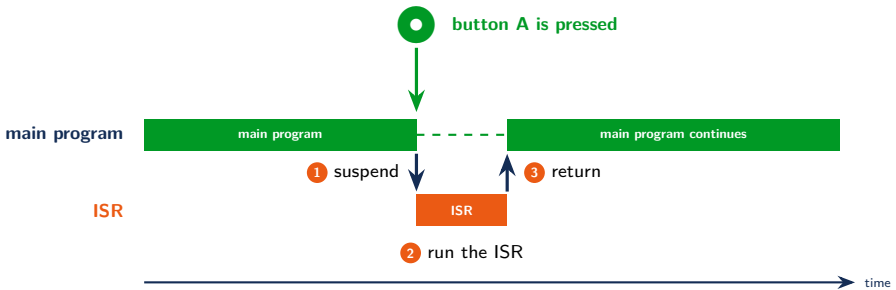
ISR = Interrupt Service Routine: a short function that the processor runs when the interrupt arrives.



2 The processor runs the ISR, for example presses++. Meanwhile the main program waits.

An interrupt: 3. return

When the ISR ends, the processor goes back to the main program, **exactly where it stopped**.



3 The main program continues. It **does not know** that it was interrupted.

Try it yourself: an interrupt

Simulators



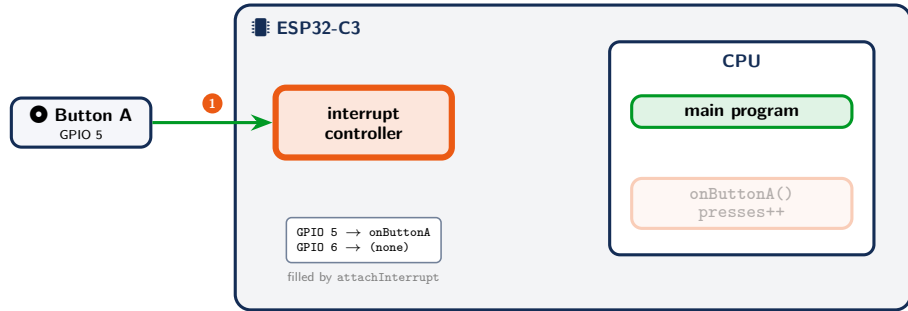
ksawerym.github.io/ComputerHardwareSimulators
topic **An interrupt**

1. Click **Start**. The main program counts: watch Work on the LCD.
2. Press button **A**. Watch the three steps: **suspend, run the ISR, return**. What happens to Work meanwhile?
3. Turn off *slow the ISR down*. How much of the CPU time does the ISR take now?

In the simulator the ISR is slowed down, so that you can see it. Really it takes a few μ s.

 ksawerym.github.io/ComputerHardwareSimulators/#cc-intr

Handling an interrupt: 1. the pin changes



- 1 Button A pulls GPIO 5 from 1 to 0 (a FALLING edge). A part of the chip, the **interrupt controller**, notices it. The CPU does not have to check anything.

The interrupt controller has a table

Interrupt	From	Function to run
GPIO 5	button A	onButtonA
GPIO 6	button B	(none)
...		

One row for each interrupt

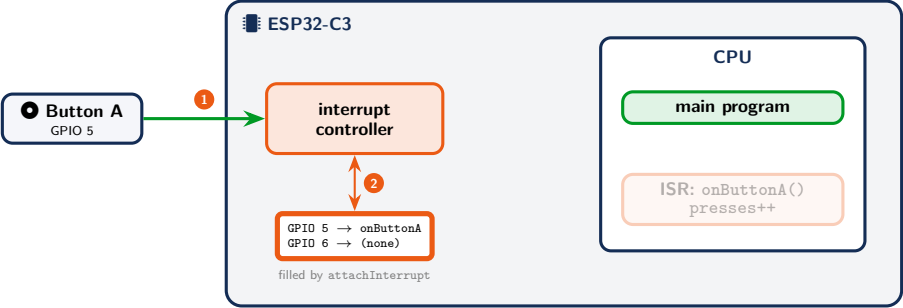
For every interrupt, the table says which function the CPU must run when it arrives.

Who fills it?

Our program, once at the start, with `attachInterrupt`.

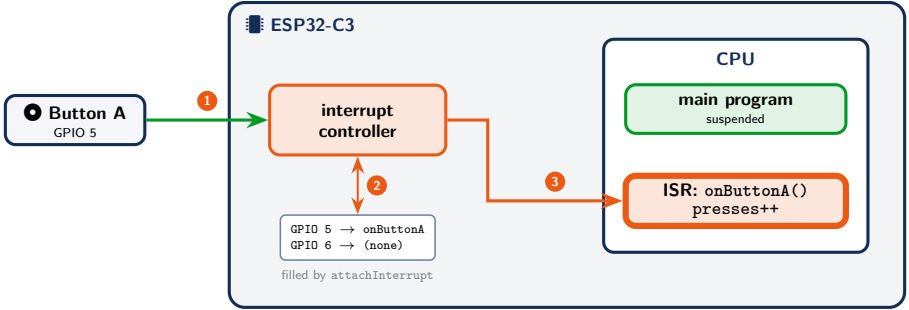
Such a function is called an **ISR**, an *Interrupt Service Routine*: the function that **handles** an interrupt. Nobody calls it in the program: the processor runs it by itself when the interrupt arrives.

Handling an interrupt: 2. which ISR?



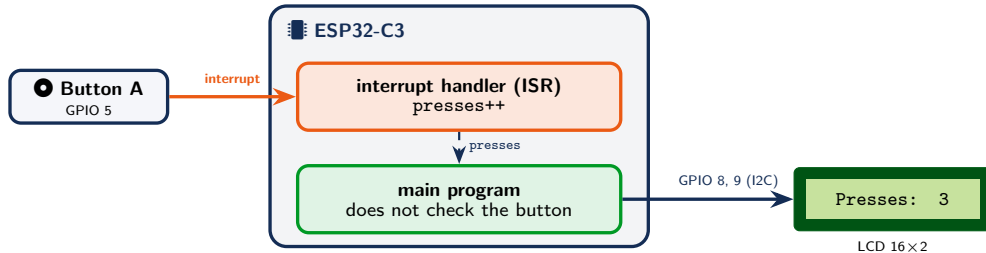
2 The interrupt controller looks up GPIO 5 in its table. The ISR for it is `onButtonA`.

Handling an interrupt: 3. the CPU runs the ISR



3 The CPU suspends the main program and runs the ISR `onButtonA: presses++`. When the ISR ends, the main program continues.

The button with an interrupt: the main program



The main program **never checks** the button. It only shows how many presses the ISR counted.

Interrupts in the code



```
const int BUTTON_A = 5;
volatile int presses = 0;

void IRAM_ATTR onButtonA() {    // the ISR
    presses++;
}

void setup() {
    pinMode(BUTTON_A, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BUTTON_A),
                    onButtonA, FALLING);
}
```

The whole program

The ISR onButtonA counts the presses. setup() connects it to button A.

Four new things

attachInterrupt, digitalPinToInterrupt, IRAM_ATTR and volatile. Let us look at them **one by one**.

Interrupts in the code: attachInterrupt

```
const int BUTTON_A = 5;
volatile int presses = 0;

void IRAM_ATTR onButtonA() { // the ISR
    presses++;
}

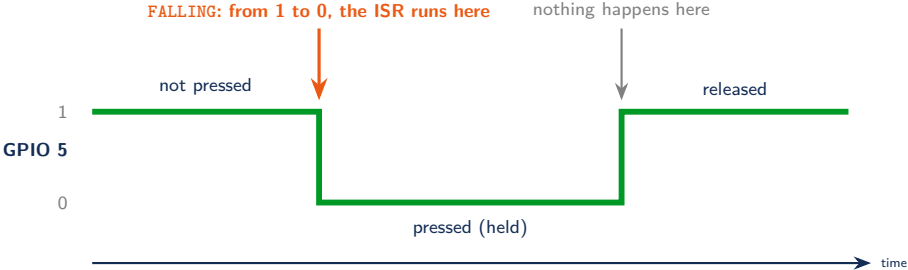
void setup() {
    pinMode(BUTTON_A, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BUTTON_A),
                   onButtonA, FALLING);
}
```

Fill the table

It writes one row into the table of the interrupt controller:
which interrupt, **which** ISR (onButtonA) and **when** (FALLING: the press).

After this line the CPU never calls onButtonA itself: the interrupt does.

Interrupts in the code: FALLING



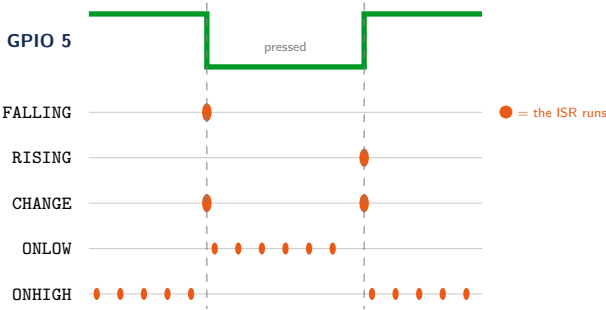
Why 1 and 0?

INPUT_PULLUP: the pin reads **1** when the button is free.
A press connects it to GND: **0**.

An edge, not a level

The ISR runs **once**, at the moment the level falls.
Holding the button longer does not count again.

Interrupts in the code: the other modes



Edges
 FALLING: 1 → 0, the press.
 RISING: 0 → 1, the release.
 CHANGE: both, so one click gives **two** interrupts.

Levels
 ONLOW, ONHIGH: the ISR runs **again and again** while the pin stays 0 (or 1).

For a button we want **one** interrupt per press: FALLING.

Interrupts in the code: digitalPinToInterrupt

```
const int BUTTON_A = 5;
volatile int presses = 0;

void IRAM_ATTR onButtonA() {    // the ISR
    presses++;
}

void setup() {
    pinMode(BUTTON_A, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BUTTON_A),
                    onButtonA, FALLING);
}
```

A pin number → an interrupt number

attachInterrupt wants the number of the **interrupt**, not of the pin. On the ESP32 both are the same; on an Arduino Uno, pin 2 is interrupt 0. With this function the code works everywhere.

Interrupts in the code: IRAM_ATTR

```

const int BUTTON_A = 5;
volatile int presses = 0;

void IRAM_ATTR onButtonA() {    // the ISR
    presses++;
}

void setup() {
    pinMode(BUTTON_A, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BUTTON_A),
                    onButtonA, FALLING);
}
    
```

Keep the ISR in fast memory

The program normally lives in the **flash**: big, slow, and sometimes busy. An ISR must start **at once**, so IRAM_ATTR puts it into **IRAM**, fast memory inside the chip.

Every ISR on the ESP32 gets IRAM_ATTR.

Interrupts in the code: volatile

```
const int BUTTON_A = 5;
volatile int presses = 0;

void IRAM_ATTR onButtonA() {    // the ISR
    presses++;
}

void setup() {
    pinMode(BUTTON_A, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(BUTTON_A),
                    onButtonA, FALLING);
}
```

Read it from memory every time

loop() never changes presses, so the compiler may read it **once** into a register and never look again. But the ISR changes it behind the program's back. volatile forbids this shortcut.

Every variable that an ISR and the main program share must be volatile.

Task 1: count the presses



```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27, 16, 2);

const int BUTTON_A = 5, BUTTON_B = 6;
volatile int presses = 0;
// TODO: the ISRs of A and B

void setup() {
  Wire.begin(8, 9);
  lcd.init(); lcd.backlight();
  // TODO: the pins and the interrupts
}

void loop() {
  // TODO: show "Presses: N"
}
```

Task

Open your project **Circuit** from Lecture 5 or the template from [GitHub](#).

1. Count the presses of button A **with an interrupt**.
2. Show the number on the LCD: Presses: N.
3. Give button B (GPIO 6) its own interrupt: its ISR sets the counter back to 0.
4. Add `delay(2000)` to `loop()`, as in Task 7 of Lecture 5. Do you still lose any presses? Why not?

Solution: count the presses



```
void IRAM_ATTR onButtonA() { presses++; }
void IRAM_ATTR onButtonB() { presses = 0; }
void setup() {
  Wire.begin(8, 9);
  lcd.init(); lcd.backlight();
  pinMode(BUTTON_A, INPUT_PULLUP);
  pinMode(BUTTON_B, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(BUTTON_A),
    onButtonA, FALLING);
  attachInterrupt(digitalPinToInterrupt(BUTTON_B),
    onButtonB, FALLING);
}

void loop() {
  lcd.setCursor(0, 0);
  lcd.print("Presses: ");
  lcd.print(presses);
  lcd.print(" ");           // clear old digits
  delay(2000);
}
```

Question 3

A second ISR, `onButtonB`, sets `presses = 0`. It is attached to pin 6 with `FALLING`, like button A.

Question 4

No press is lost. The interrupt counts it even while `loop()` waits. The LCD just shows the new number up to 2 s later.

Compare with Task 7

The same `delay(2000)`, but now nothing is missed.

Two buttons at the same moment?

✓ **On our ESP32-C3: safe**

It has **one** core, and an ISR is not interrupted by another button. If A and B are pressed together, one ISR runs, then the other. Both presses are handled.

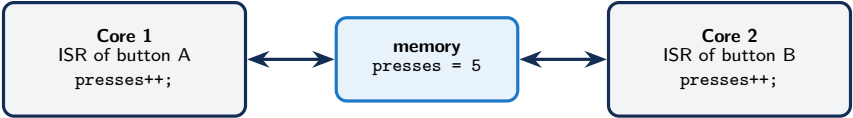
✓ **Our Task 1: safe**

loop() only **reads** presses. Only the ISRs change it.

But what if the processor had **two** cores?

What if we had two cores?

⚡ both buttons at the same moment



Suppose
Both buttons now count presses, and each ISR runs on **its own core**. Both buttons are pressed at the same moment, so both ISRs run **at the same time**.

presses was 5. Two presses. What should it be after both ISRs?

Two cores: a press is lost

	presses
before	5
expected	7
sometimes we get	6



One press is lost

Two cores ran presses++ at the same moment, and the counter grew by **one**, not by two. Not every time: only when the timing is unlucky.

How can one line of code lose a press?

Keep this question. We meet it again at the end of today, and answer it on Tuesday.

The real danger: an ISR and loop() change one variable

loop() takes the presses and resets the counter:

```
int n = presses;    // read
presses = 0;        // reset
```

⚠ A press between these two lines

The ISR adds 1 to presses. Then loop() sets 0 over it.
The press is **lost**, even on one core.

Two pieces of code change one variable: a **race**. We meet it again at the end of today, and solve it on Tuesday.

Rules for an ISR: 1. keep it short

short ISR



long ISR



While the ISR runs

The main program is **suspended**. A long ISR stops the whole program.

Other interrupts wait too

A second press that comes during a long ISR is handled only **after** it.

A good ISR takes **microseconds**: presses++ and nothing more.

Rules for an ISR: 2. some things are not allowed

✗ Not allowed: the ESP32 crashes and restarts

```

void IRAM_ATTR onButtonA() {
  presses++;
  delay(100);           // waits
  Serial.println(presses); // waits
  lcd.print(presses);   // waits
}

```

These functions wait inside

delay() waits for the time to pass. Serial and the LCD wait until the previous data has been sent. To wait, they ask the system to put the code to sleep and come back later.

✓ Allowed

```

void IRAM_ATTR onButtonA() {
  presses++;           // only record it
}
// loop() prints presses on the LCD

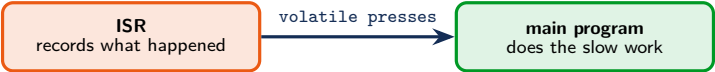
```

An ISR must never wait

An ISR cannot be put to sleep: there is nothing to come back to it. On the ESP32 the program **crashes** and the board restarts.

Rule 1 is about **time**: be short. Rule 2 is about **what**: never call a function that waits.

Rules for an ISR: 3. share data through volatile variables



The ISR writes
It only changes a variable: `presses++`.

loop() reads
It shows the value on the LCD, sends it, does the slow work. `volatile`: it reads the **fresh** value every time.

The ISR only **records** what happened. The main program **does the work**.

You can use interrupts. How does the processor do it?



Where to come back?

How does the processor know where the main program stopped?



The same registers

The ISR uses registers too. Why does the main program not notice?



When exactly?

Does the processor stop in the middle of an instruction?

Now we look **inside the processor**: what happens there, step by step.

A register has two names

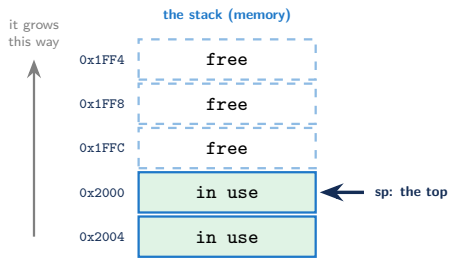
Number	Name	Used for
x2	sp	the stack pointer
x5	t0	temporary values
x6	t1	temporary values
x10	a0	arguments (addresses)
x11	a1	arguments (addresses)
x14	a4	arguments, temporary values
x15	a5	arguments, temporary values

The same register

t0 is x5. The processor only knows the number; the name tells people what the register is for. From now on we use the names.

`addi t0, t0, 1` is exactly `addi x5, x5, 1`.

The stack



The **stack** is a part of the memory where a program keeps values **for later**, for example the local variables of a function.

The top
The register *sp* (*stack pointer*) holds the address of the top. From the top down: **in use**. Above it: free.

It grows towards lower addresses
More values: *sp* goes **down**. Fewer: *sp* goes up.

A local variable lives on the stack

C



```
int f(void) {
    int a = 7;
    return a + 1;
}
```

One local variable

a exists only while f runs. Where does it live?

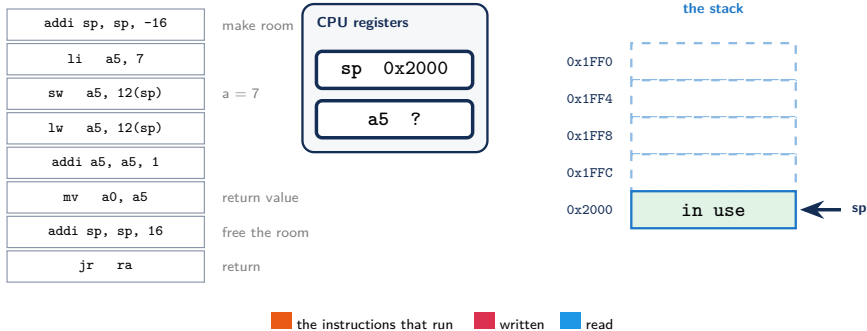
RISC-V (gcc -O0 -fomit-frame-pointer)



```
f:
    addi sp, sp, -16    # make room
    li   a5, 7
    sw   a5, 12(sp)    # a = 7, on the stack
    lw   a5, 12(sp)    # read a
    addi a5, a5, 1
    mv   a0, a5        # the return value
    addi sp, sp, 16    # free the room
    jr   ra            # return
```

The function takes room on the stack at the start, and gives it back at the end.

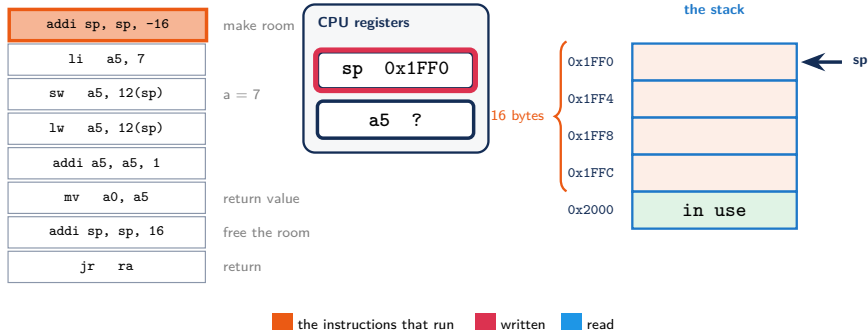
A local variable lives on the stack: step by step



Before f

`sp = 0x2000`. Above it the stack is free.

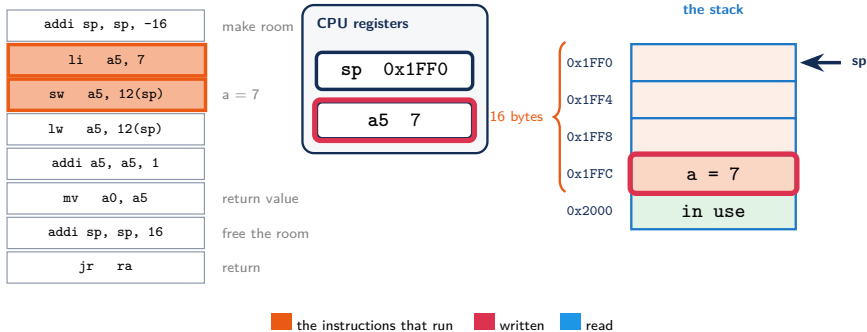
A local variable lives on the stack: step by step



`addi sp, sp, -16`

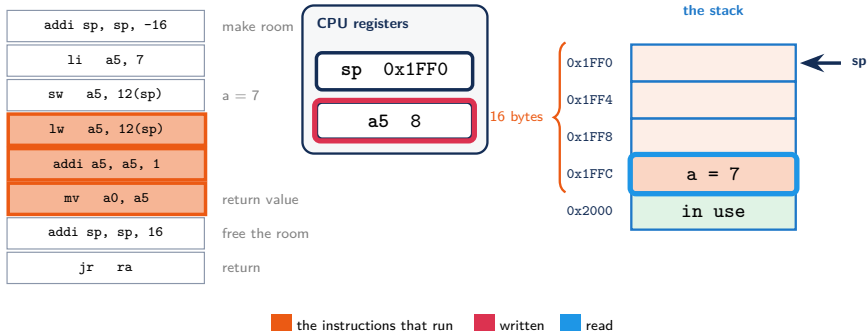
The stack **grows**: 16 bytes for `f`, `sp = 0x1FF0`. (RISC-V keeps `sp` a multiple of 16, so 16 and not 4.)

A local variable lives on the stack: step by step



`li a5, 7` and `sw a5, 12(sp)`
`a = 7` is written at `sp + 12 = 0x1FFC`.

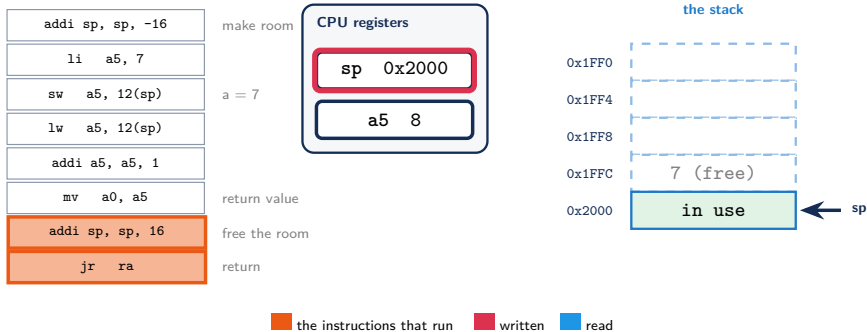
A local variable lives on the stack: step by step



`lw, addi, mv`

f reads a from the stack, adds 1 and puts the result (8) in a0.

A local variable lives on the stack: step by step



`addi sp, sp, 16`

The stack **shrinks**: `sp = 0x2000` again. `a` is gone: the `7` is still in the memory, but that place is free now.

Task: find the stack in Compiler Explorer



```
int sum3(int a, int b, int c) {
    int s = a + b;
    int t = s + c;
    return t;
}
```

Settings

godbolt.org, language C, compiler **RISC-V**
(32-bits) gcc, options -O0
-fomit-frame-pointer.

Task

Compile the function and read its assembly.

1. Which instruction makes room on the stack? How many bytes?
2. a, b, c arrive in a0, a1, a2. Where on the stack are they written?
3. Where are s and t kept?
4. Which instruction frees the room?

If you finish early: change -O0 to -O2. What happens to the stack?

Solution: find the stack in Compiler Explorer

●●●

```
sum3:
    addi sp, sp, -32
    sw   a0, 12(sp)    # a
    sw   a1, 8(sp)     # b
    sw   a2, 4(sp)     # c
    lw   a4, 12(sp)
    lw   a5, 8(sp)
    add  a5, a4, a5
    sw   a5, 28(sp)    # s
    lw   a4, 28(sp)
    lw   a5, 4(sp)
    add  a5, a4, a5
    sw   a5, 24(sp)    # t
    lw   a5, 24(sp)
    mv   a0, a5
    addi sp, sp, 32
    jr   ra
```

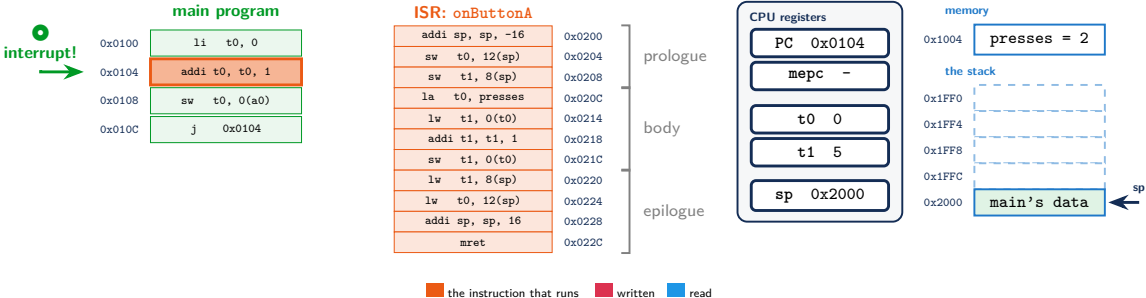
Questions 1 and 4
addi sp, sp, -32 makes room for **32 bytes**; addi sp, sp, 32 frees it.

Questions 2 and 3		
Variable	Comes from	On the stack
a	a0	12(sp)
b	a1	8(sp)
c	a2	4(sp)
s	a + b	28(sp)
t	s + c	24(sp)

Your offsets may differ a little.

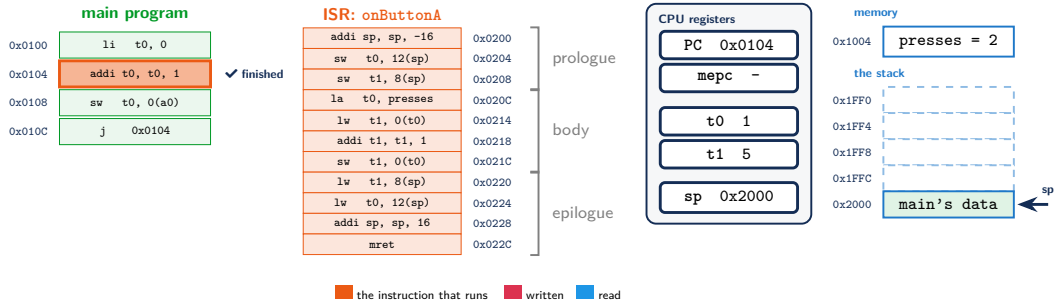
If you finished early
With -O2: add a0, a0, a1, add a0, a0, a2, ret. No stack at all: the values stay in registers.

Inside the CPU: 1. the interrupt arrives



1 The signal
 Button A raises an interrupt signal while the CPU runs addi at 0x0104.

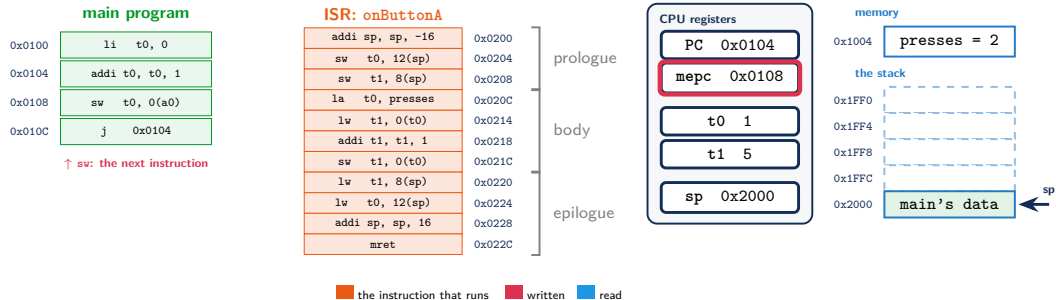
Inside the CPU: 1. the interrupt arrives



2 Finish the instruction

The CPU does not stop in the middle: it first **finishes** addi. Now $t0 = 1$.

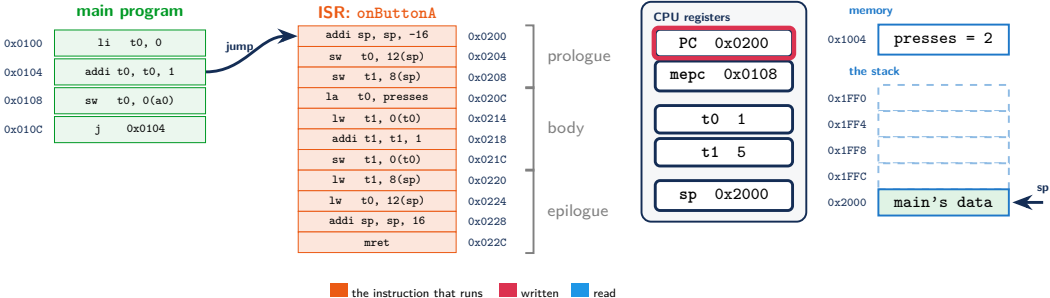
Inside the CPU: 1. the interrupt arrives



3 Remember the way back

The CPU writes into mepc the address of the **next** instruction, sw, at 0x0108. The instruction addi has already been executed.

Inside the CPU: 1. the interrupt arrives

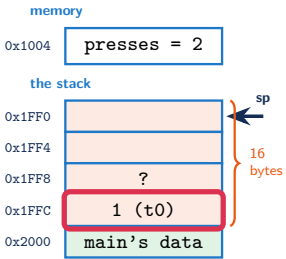
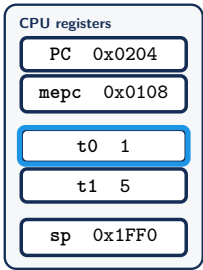


4 Jump
The PC gets the address of the ISR: 0x0200. The ISR starts with its **prologue**.

Inside the CPU: 2. the prologue

main program
paused
at 0x0108

ISR: onButtonA		
addi sp, sp, -16	0x0200	prologue
sw t0, 12(sp)	0x0204	
sw t1, 8(sp)	0x0208	
la t0, presses	0x020C	body
lw t1, 0(t0)	0x0214	
addi t1, t1, 1	0x0218	
sw t1, 0(t0)	0x021C	
lw t1, 8(sp)	0x0220	
lw t0, 12(sp)	0x0224	epilogue
addi sp, sp, 16	0x0228	
mret	0x022C	



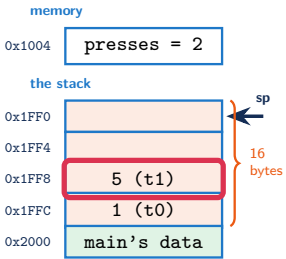
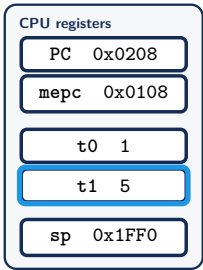
the instruction that runs written read

sw t0, 12(sp)
Save the main program's t0 (1): the body is about to use t0.

Inside the CPU: 2. the prologue

main program
paused
at 0x0108

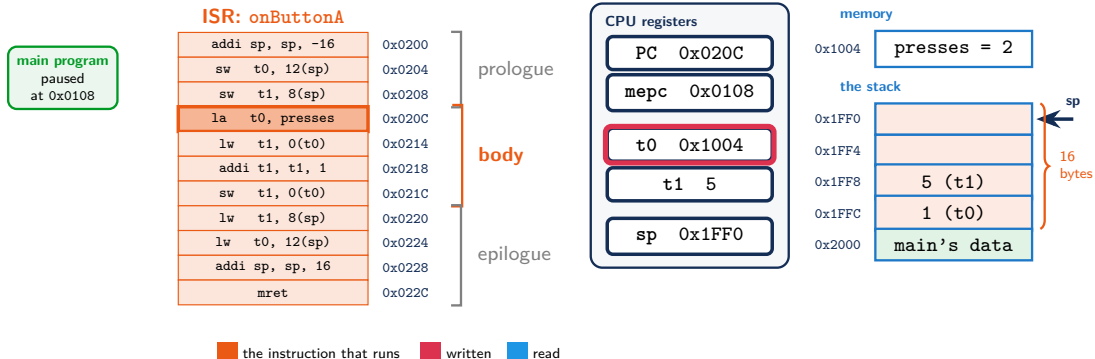
ISR: onButtonA		
addi sp, sp, -16	0x0200	prologue
sw t0, 12(sp)	0x0204	
sw t1, 8(sp)	0x0208	
la t0, presses	0x020C	body
lw t1, 0(t0)	0x0214	
addi t1, t1, 1	0x0218	
sw t1, 0(t0)	0x021C	
lw t1, 8(sp)	0x0220	epilogue
lw t0, 12(sp)	0x0224	
addi sp, sp, 16	0x0228	
mret	0x022C	



the instruction that runs written read

sw t1, 8(sp)
Save the main program's t1 (5): the body uses t1 too.

Inside the CPU: 3. the body



la t0, presses

t0 = the **address** of presses: 0x1004. The ISR cannot know what the main program keeps in its registers, so it gets the address itself. (1a is two real instructions, 8 bytes: the next one is at 0x0214.)

Inside the CPU: 3. the body

main program
paused
at 0x0108

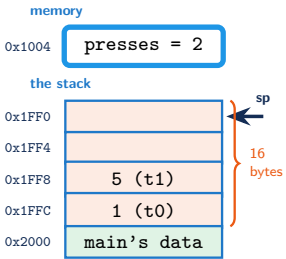
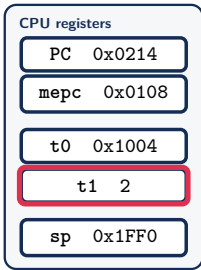
ISR: onButtonA

addi sp, sp, -16	0x0200
sw t0, 12(sp)	0x0204
sw t1, 8(sp)	0x0208
la t0, presses	0x020C
lw t1, 0(t0)	0x0214
addi t1, t1, 1	0x0218
sw t1, 0(t0)	0x021C
lw t1, 8(sp)	0x0220
lw t0, 12(sp)	0x0224
addi sp, sp, 16	0x0228
mret	0x022C

prologue

body

epilogue



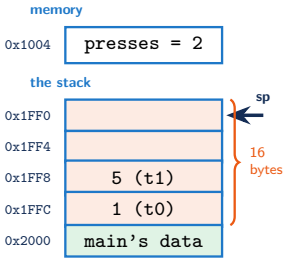
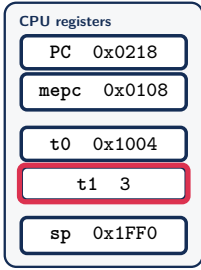
the instruction that runs written read

lw t1, 0(t0)
t1 = presses = 2.

Inside the CPU: 3. the body

main program
paused
at 0x0108

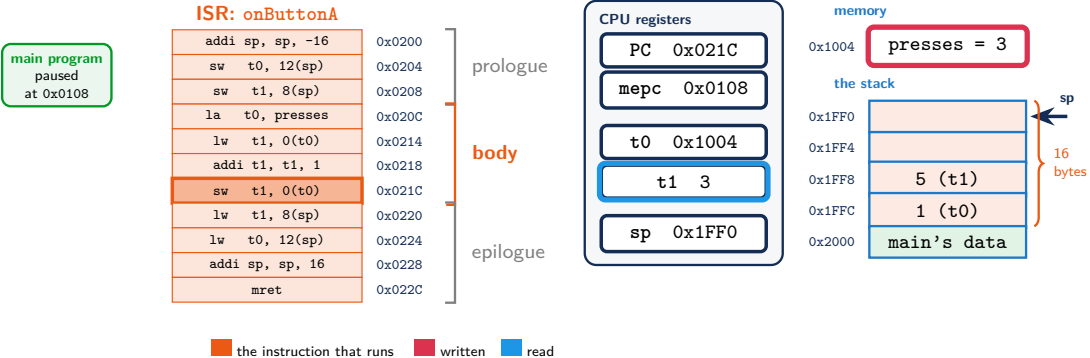
ISR: onButtonA		
addi sp, sp, -16	0x0200	prologue
sw t0, 12(sp)	0x0204	
sw t1, 8(sp)	0x0208	
la t0, presses	0x020C	
lw t1, 0(t0)	0x0214	body
addi t1, t1, 1	0x0218	
sw t1, 0(t0)	0x021C	
lw t1, 8(sp)	0x0220	
lw t0, 12(sp)	0x0224	epilogue
addi sp, sp, 16	0x0228	
mret	0x022C	



the instruction that runs written read

addi t1, t1, 1
t1 = 3.

Inside the CPU: 3. the body

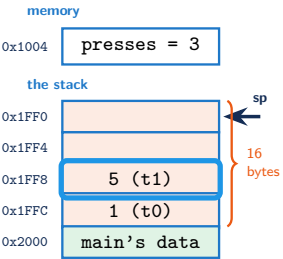
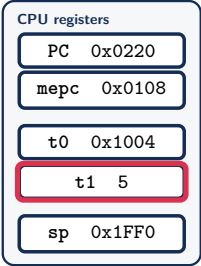


sw t1, 0(t0)
presses = 3. These four instructions are presses++: the work of the ISR.

Inside the CPU: 4. the epilogue and return

main program
paused
at 0x0108

ISR: onButtonA		
addi sp, sp, -16	0x0200	prologue
sw t0, 12(sp)	0x0204	
sw t1, 8(sp)	0x0208	
la t0, presses	0x020C	
lw t1, 0(t0)	0x0214	body
addi t1, t1, 1	0x0218	
sw t1, 0(t0)	0x021C	
lw t1, 8(sp)	0x0220	
lw t0, 12(sp)	0x0224	epilogue
addi sp, sp, 16	0x0228	
mret	0x022C	



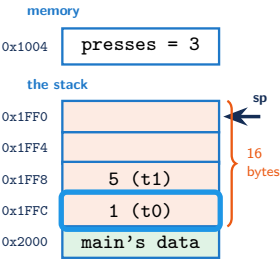
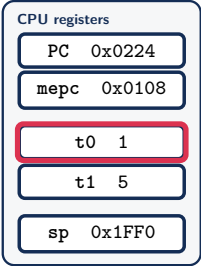
the instruction that runs written read

lw t1, 8(sp)
Restore the main program's t1: 5.

Inside the CPU: 4. the epilogue and return

main program
paused
at 0x0108

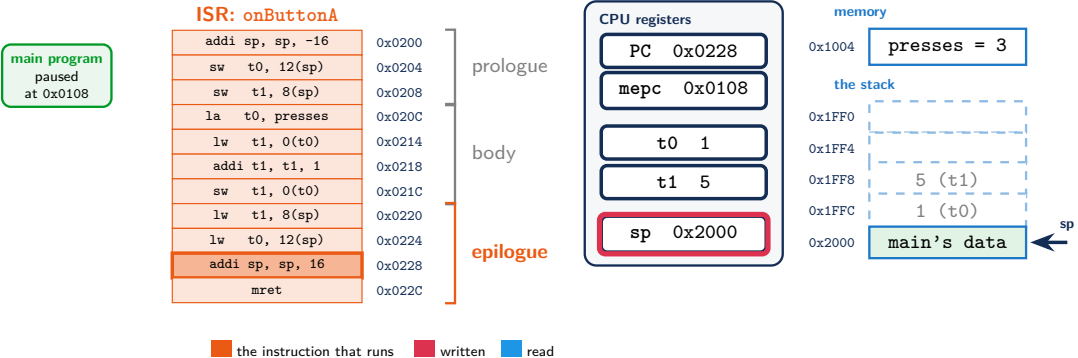
ISR: onButtonA		
addi sp, sp, -16	0x0200	prologue
sw t0, 12(sp)	0x0204	
sw t1, 8(sp)	0x0208	
la t0, presses	0x020C	body
lw t1, 0(t0)	0x0214	
addi t1, t1, 1	0x0218	
sw t1, 0(t0)	0x021C	
lw t1, 8(sp)	0x0220	
lw t0, 12(sp)	0x0224	epilogue
addi sp, sp, 16	0x0228	
mret	0x022C	



the instruction that runs written read

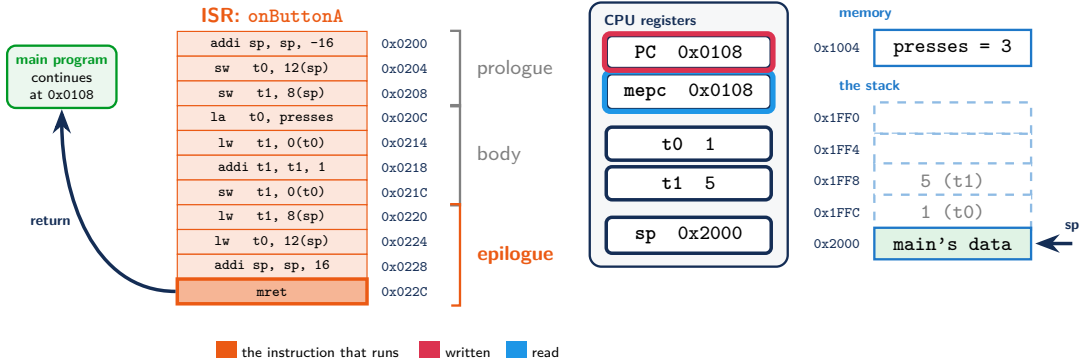
lw t0, 12(sp)
Restore the main program's t0: 1.

Inside the CPU: 4. the epilogue and return



addi sp, sp, 16
Free the room: sp is 0x2000 again. The stack is as before the interrupt.

Inside the CPU: 4. the epilogue and return



```
mret
```

PC $\leftarrow$ mepc = 0x0108. The main program continues with sw, with t0 = 1 and t1 = 5: it **does not know** that it was interrupted.

The rule: save what you change

The hardware saves **only the PC** (in `mepc`). Every register that the ISR changes, the ISR must **save** in its prologue and **restore** in its epilogue.

Our ISR

Its body changes `t0` and `t1`. So the prologue saves exactly these two: that is enough.

sp?

It changes too, but the epilogue moves it back by the same 16 bytes.

Then the main program finds all its registers exactly as it left them.

But our ISR is written in C

We write



```
void IRAM_ATTR onButtonA() {
    presses++;
}
```

We write only the body

Which registers will the compiled code use?

We do not decide it, and we do not see it.

The compiler makes (gcc -O2)



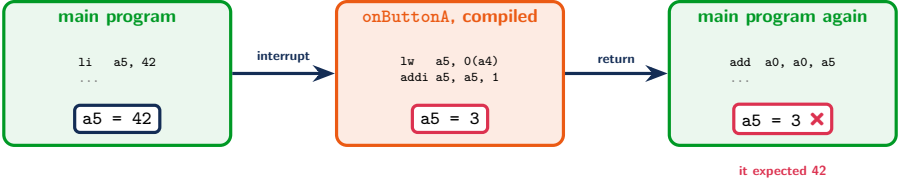
```
onButtonA:
    la    a4, presses    # the address
    lw    a5, 0(a4)
    addi  a5, a5, 1
    sw    a5, 0(a4)
    ret                          # a normal return
```

A normal function

No prologue, no epilogue. It changes a4 and a5 and **saves neither**. And it ends with ret, not mret.

Is that a mistake of the compiler? No. Let us see why, and why it is still a problem for an ISR.

The problem: nobody saves a5



The rule of C

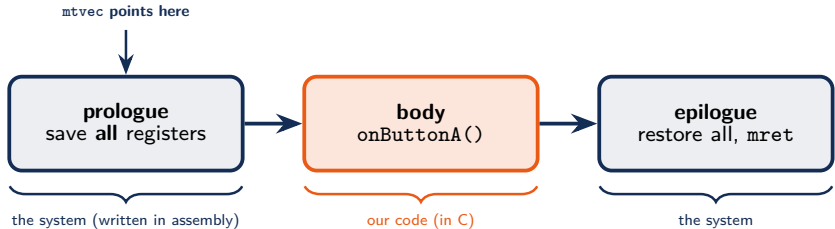
A function **may** change t0 to t6 and a0 to a7 without saving them. If the caller still needs one of them, the **caller** saves it before the call.

But nobody called the ISR

The main program was **interrupted**, between any two instructions. It had no chance to save a5. And the C function does not save it either.

So somebody must save the registers **before** our C function runs, and restore them after it.

The interrupt entry code



Always all registers

The system does not know which registers our C function will use. So it **always** saves all of them, a4 and a5 too, for every interrupt.

The cost

29 registers: 29 sw and 29 lw, about a third of a microsecond at 160 MHz. This code also finds the right ISR in the table.

Who saves a5? This code, before our ISR runs: it saves **every** register.

The interrupt entry code: the real one



```

_interrupt_handler:
    addi sp, sp, -128    # room for 32 registers
    sw   ra, 4(sp)
    sw   t0, 20(sp)
    sw   a5, 60(sp)
    ...                  # 29 sw in total
    csrr t0, mepc        # mepc too
    sw   t0, 0(sp)
    call rtos_int_enter
    jal  _global_interrupt_handler
    call rtos_int_exit
    lw   ra, 4(sp)
    ...                  # 29 lw in total
    addi sp, sp, 128
    mret
    
```

Is the picture simplified?

Not much. This is the real code of the ESP32-C3 libraries (ESP-IDF), shortened only at the dots.

Prologue and epilogue

All registers and mepc go to the stack. At the end: all back, and mret.

The body

_global_interrupt_handler looks into the table and calls our onButtonA with a normal call. rtos_int_enter and exit tell FreeRTOS that an interrupt runs.

Remember: **all** registers of the interrupted program are on its stack. The scheduler needs this.

Can an ISR be faster?

```
void __attribute__((interrupt))
onButtonA() {
    presses++;
}
```

The compiler makes (gcc -O2)

```
addi sp, sp, -16
sw    a4, 12(sp)    # only a4
sw    a5, 8(sp)     # and a5
...    # presses++
lw    a4, 12(sp)
lw    a5, 8(sp)
addi  sp, sp, 16
mret
```

Save only what you use

With interrupt, the compiler writes the prologue and the epilogue itself, exactly like our ISR in assembly: 2 registers instead of 29, and mret at the end. An ISR written by hand in assembly can do the same.

When is it worth it?

When interrupts come **very often**, for example 100 000 times per second to control a motor or to read sound samples. Then every saved instruction counts.

⚠ The price

Such an ISR skips the system: it cannot use FreeRTOS functions or wake a task, and it works on one chip only. The Arduino core does not even offer it: there the entry code always runs first.

Simulator 1: polling and interrupts

Simulator task

Open **ComputerHardwareSimulators**, topic **Polling vs interrupt**.

- 1. Mode *2 s of work, then one check*. Click **Start**, then **5 short clicks**. How many presses does polling count? And the interrupt?
- 2. Look at the row *CPU: interrupts*. How much of the CPU time went to the ISR?

Then

Topic **Interrupt step by step**. The signal from button A arrives **during** the instruction at 0x0108.

● ● ●

```
0x0100  li    t0, 0
0x0104  addi  t0, t0, 1
0x0108  sw    t0, 0(a0)
0x010C  j     0x0104
```

- 3. **Predict** the saved PC. Then step through and check.
- 4. The ISR uses t0 and t1. What are they in the main program after the return?

 [ksawerym.github.io/ComputerHardwareSimulators/#cc-poll](https://github.com/ksawerym/ComputerHardwareSimulators/#cc-poll) [ksawerym.github.io/ComputerHardwareSimulators/#cc-irq](https://github.com/ksawerym/ComputerHardwareSimulators/#cc-irq)

Solution: Simulator 1

Question 1

Polling: usually **0 or 1**. A click counts only if it happens exactly during a check. Interrupt: **5**.

Question 2

Almost **nothing**. Each ISR takes a few microseconds: on the timeline it is only a thin orange line.

Question 3

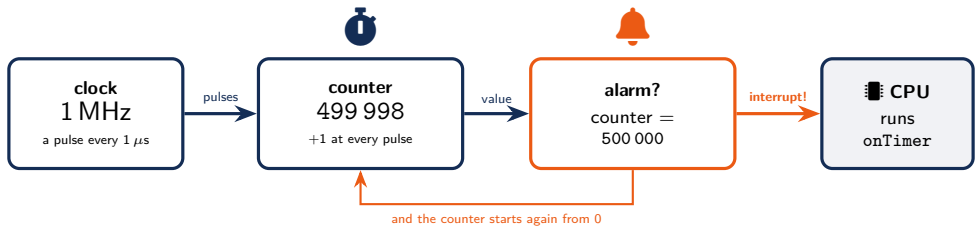
0x010C. The CPU first finishes sw, so the saved PC is the address of the **next** instruction.

Question 4

The same as before the interrupt: $t_0 = 1$, $t_1 = 5$. The ISR saved both on the stack and restored them before `mret`.

An interrupt misses nothing, costs almost no CPU time, and the main program continues **exactly where it stopped**.

A timer is a stopwatch with an alarm



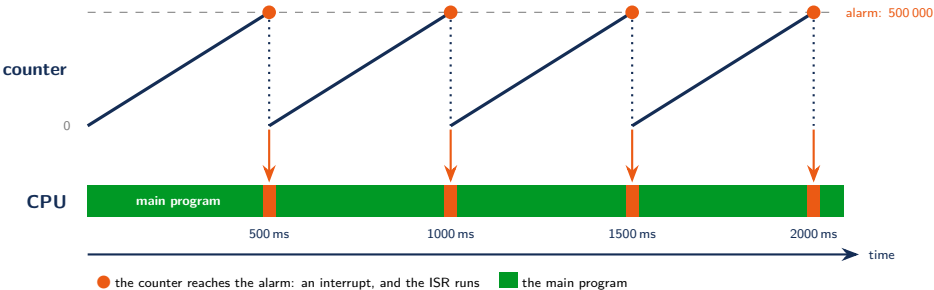
It counts by itself

A **timer** is a counter in hardware. A clock gives it pulses at a fixed rate, and at every pulse it adds 1. The CPU does not have to do anything.

It rings

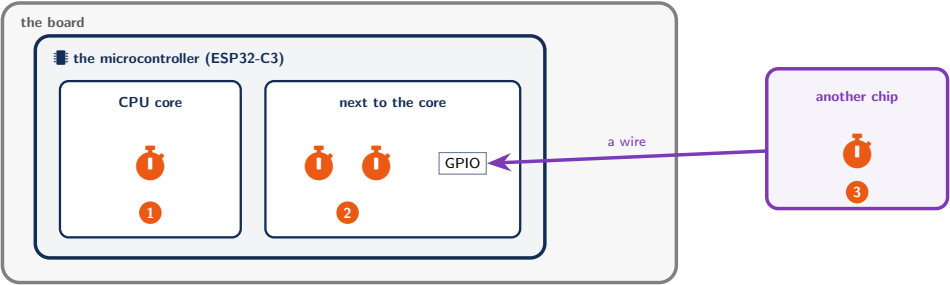
When the counter reaches the set value (the **alarm**), the timer raises an **interrupt**. Then it can start again from 0, and ring again.

Interrupts from a timer



A timer interrupt lets the processor do something **regularly**, without checking the time in the main program.

Where is the timer?



1 In the core

Many processors have a timer in the core itself: every core of a PC has one, and RISC-V defines one (`mtime`). The operating system takes its regular interrupt from it.

2 Next to the core

A device on the same chip, like the GPIO controller. The ESP32-C3 has **two** such timers for us, and one more for the system.

3 On another chip

For example a real-time clock with an alarm. Its signal comes on a wire to a GPIO pin: for the CPU it is a GPIO interrupt, just like button A.

A timer in the code



```

const int LED = 4;
hw_timer_t *timer = nullptr;
volatile bool ledOn = false;

void IRAM_ATTR onTimer() {           // the ISR
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
}

void setup() {
    pinMode(LED, OUTPUT);
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTimer);
    timerAlarm(timer, 500000, true, 0);
}
    
```

The whole program

The ISR onTimer switches the LED. setup() starts a timer and connects it to onTimer.

Three new things

timerBegin, timerAttachInterrupt and timerAlarm: the clock, the ISR and the alarm of our stopwatch. One by one.

A timer in the code: timerBegin

● ● ●

```

const int LED = 4;
hw_timer_t *timer = nullptr;
volatile bool ledOn = false;

void IRAM_ATTR onTimer() {           // the ISR
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
}

void setup() {
    pinMode(LED, OUTPUT);
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTimer);
    timerAlarm(timer, 500000, true, 0);
}
    
```

Take a timer and set its clock

It takes a free hardware timer and sets its clock to **1 000 000** pulses per second: the counter grows by 1 every 1 μ s.

It returns the timer. We keep it in timer, of the type hw_timer_t *.

A timer in the code: timerAttachInterrupt



```
const int LED = 4;
hw_timer_t *timer = nullptr;
volatile bool ledOn = false;

void IRAM_ATTR onTimer() {           // the ISR
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
}

void setup() {
    pinMode(LED, OUTPUT);
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTimer);
    timerAlarm(timer, 500000, true, 0);
}
```

Which ISR?

Just like attachInterrupt for the button: it writes onTimer into the table, for the interrupt of **this** timer.

No pin and no mode: a timer has only one event, the alarm.

A timer in the code: timerAlarm

```

const int LED = 4;
hw_timer_t *timer = nullptr;
volatile bool ledOn = false;

void IRAM_ATTR onTimer() {           // the ISR
    ledOn = !ledOn;
    digitalWrite(LED, ledOn);
}

void setup() {
    pinMode(LED, OUTPUT);
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTimer);
    timerAlarm(timer, 500000, true, 0);
}

```

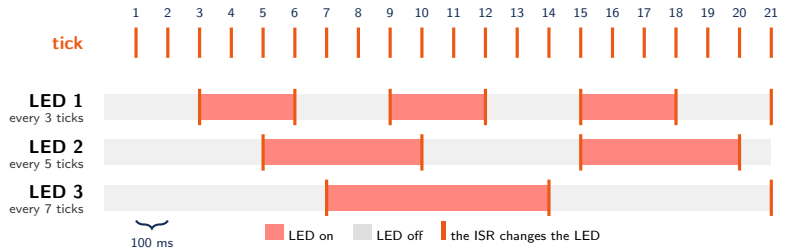
When does it ring?

500000: when the counter reaches 500 000 steps of 1 μ s, so after **500 ms**.
 true: then start again from 0.
 0: and repeat without a limit.

The LED blinks with **no code in loop()**: the timer calls onTimer every 500 ms.

Three LEDs, three rhythms, one timer

Three LEDs should change every **300**, **500** and **700 ms**. But the ESP32-C3 has only **two** hardware timers.



One timer interrupt at a fixed rate is called a **tick**. Here: a tick every 100 ms. At every tick, the ISR **decides** what to do: LED 1 every 3 ticks, LED 2 every 5, LED 3 every 7.

The tick in the code



```
volatile unsigned long ticks = 0;
volatile bool led1 = false, led2 = false, led3 = false;

void IRAM_ATTR onTick() {           // every 100 ms
    ticks++;
    if (ticks % 3 == 0) { led1 = !led1; digitalWrite(LED1, led1); }
    if (ticks % 5 == 0) { led2 = !led2; digitalWrite(LED2, led2); }
    if (ticks % 7 == 0) { led3 = !led3; digitalWrite(LED3, led3); }
}
```

`ticks % 3 == 0`
Every third tick: 300 ms.

One timer
`timerAlarm(timer, 100000, true, 0):`
a tick every 100 ms.

Remember this idea: a **regular tick**, and a **decision** at every tick. We will need it in Part 2.

Simulator 2: the tick

Simulator task

Topic **Timer and tick**: a tick every 100 ms. LED 1 changes every 3 ticks, LED 2 every 5, LED 3 every 7. At the start all LEDs are off. **Predict first**, then check.

1. Which LEDs change at tick **15**? At tick **21**? At tick **35**?
2. Which LEDs are **on** right after tick **10**?
3. At which tick do all three change **together** for the first time? After how many seconds?

	LED 1	LED 2	LED 3
changes at tick 15	...	...	...
changes at tick 21	...	...	...
changes at tick 35	...	...	...
on after tick 10	...	...	...

 [ksawerym.github.io/ComputerHardwareSimulators/#cc-tick](https://github.com/Ksawerym/ComputerHardwareSimulators/#cc-tick)

Solution: Simulator 2

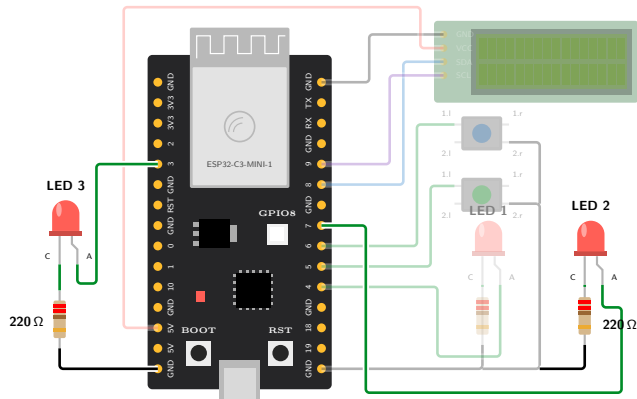
	LED 1	LED 2	LED 3
changes at tick 15	✓	✓	-
changes at tick 21	✓	-	✓
changes at tick 35	-	✓	✓
on after tick 10	on	off	on

Question 2
 LED 1 changed 3 times (ticks 3, 6, 9): on. LED 2 twice (5, 10): off. LED 3 once (7): on.

Question 3
 At tick $3 \times 5 \times 7 = 105$, after **10.5 s**.

A LED changes when the tick number is divisible by its period: the ISR checks this at every tick.

Build it: two more LEDs



- 1

Parts

Click +, add two **LEDs** and two **Resistors** (220 Ω).
- 2

Wires

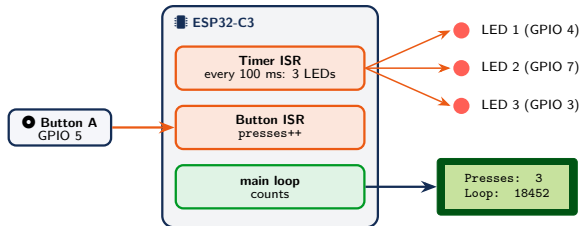
GPIO 7 $\rightarrow$ LED 2 A. GPIO 3 $\rightarrow$ LED 3 A. Each C $\rightarrow$ resistor $\rightarrow$ **GND**.
- 3

Keep it

The LED on GPIO 4 is now **LED 1**. Save the project.

Our circuit now: LEDs on GPIO 4, 7 and 3; buttons on 5 and 6; LCD on 8 and 9.

Task 2: three LEDs



Task

Use your circuit with three LEDs or the template from [GitHub](#).

1. Make LED 1 blink every 500 ms **with a timer interrupt**, as on the slide “A timer in the code”.
2. Change it: one tick every **100 ms**, and the three LEDs change every 300, 500 and 700 ms.
3. Keep the button counter from Task 1. In `loop()`, also count as fast as possible and show Loop: N in row 1. Do the LEDs keep their rhythm?

If you finish early: make LED 1 blink with `millis()` in `loop()` instead of the timer, and keep the LCD writes. Is its rhythm still even?

Solution: three LEDs



```
const int LED1 = 4, LED2 = 7, LED3 = 3;
long loops = 0;
void setup() {
    // the LCD and button A as in Task 1
    pinMode(LED1, OUTPUT);  pinMode(LED2, OUTPUT);
    pinMode(LED3, OUTPUT);
    timer = timerBegin(1000000);
    timerAttachInterrupt(timer, &onTick);
    timerAlarm(timer, 100000, true, 0); // 100 ms
}

void loop() {
    loops++;
    lcd.setCursor(0, 0);
    lcd.print("Presses: ");  lcd.print(presses);
    lcd.setCursor(0, 1);
    lcd.print("Loop: ");    lcd.print(loops);
}
```

The ISR

onTick from the slide “The tick in the code”.

Question 3

Yes. The LEDs and the button are handled by interrupts, so it does not matter what loop() does, even slow LCD writes.

Simulator 3: could we do this without interrupts?

With `delay()`

```

● ● ●
void loop() {
    digitalWrite(LED1, HIGH); delay(300);
    digitalWrite(LED1, LOW);  delay(300);
    // and LED 2? LED 3?
}
```

With `millis()`

```

● ● ●
void loop() {
    if (millis() - last1 >= 300) {
        last1 += 300; toggle(LED1);
    }
    // the same for LED 2 and LED 3
    showOnLcd();           // several ms
}
```

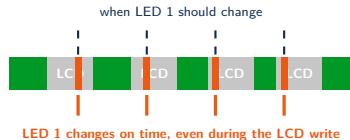
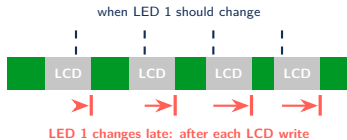
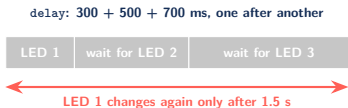
Simulator task

Topic **Timer and tick**, switch *How*: `delay()`, `millis()` in `loop()` or a timer interrupt. Turn on *loop()* writes to the LCD.

1. `delay()`: do the LEDs keep 300, 500 and 700 ms?
2. `millis()` in `loop()`: look at the moments when LED 1 changes. Are they on time?
3. Timer interrupt: are they on time now?

ksawerym.github.io/ComputerHardwareSimulators/#cc-tick

Solution: Simulator 3



1. delay()

No. While the CPU waits for one LED, it cannot serve the others. The waits add up.

2. `millis()`

Almost. But each slow step in the loop delays the LEDs: the rhythm is **uneven**.

3. Timer interrupt

Yes. The tick comes on time, whatever the loop is doing.

`millis()` in `loop()` is polling again: the loop keeps asking “is it time yet?”.

Without interrupts it is possible, but it is **hard to keep an exact rhythm** while the program also does other work.

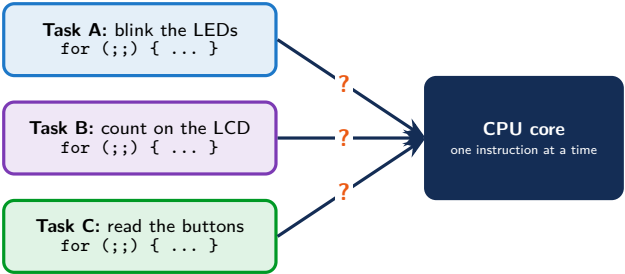
Interrupts: summary

Polling	Interrupts
the processor asks the device	the device tells the processor
short events can be missed	nothing is missed
wastes time on checking	the processor works until something happens
a rhythm is hard to keep	a timer gives an exact rhythm

Our tick decided **which LED** to change. Next: what if the tick decided **which program** runs?

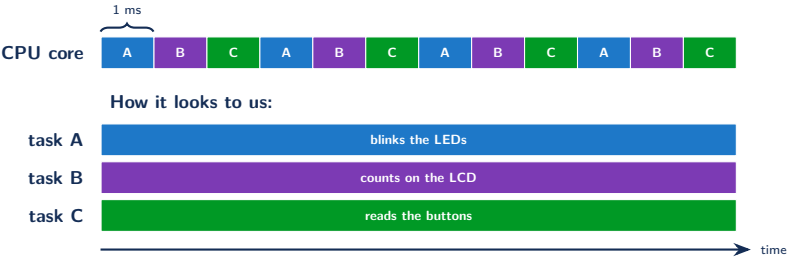
One processor, several tasks

In Task 2 each LED needed only a tiny piece of work, so it fit into the ISR. But real tasks are **long programs** with their own loops and waits: read a sensor, update the LCD, talk over Wi-Fi. They cannot run inside a short ISR.



How can three programs, each with its own endless loop, share **one core**?

The idea: take turns very quickly



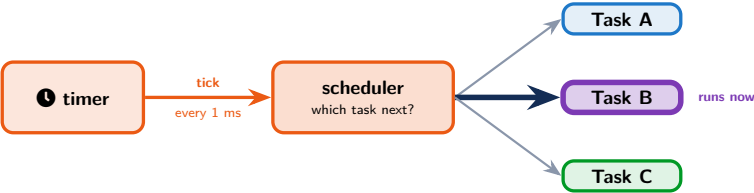
Time slice
 Each task runs for a short **time slice**, then the next one gets the core. The switches are so fast that all tasks seem to run at the same time.

Concurrency: several tasks make progress by **taking turns** on one core.

Several cores working at the same moment: *parallelism*. More on Tuesday.

Who decides when to switch?

The **scheduler** is the part of the system that decides **which task runs next** and switches to it.



Task 2
tick → **which LED?**

The scheduler
tick → **which task?**

The same idea as in Task 2: a regular **tick**, and a decision at every tick. On our ESP32 the tick comes every **1 ms**.

Simulator 4: time slices

Simulator task

Topic **Time slice**: tasks A, B and C take turns (round robin), one time slice = 1 ms. Task A starts at 0 ms. **Predict first**, then check.

1. Which task runs from **7 to 8 ms**?
2. How many task switches happen in **one second**?
3. Set the time slice to **500 ms**. Look at the row *how it looks to us*. Do the tasks still seem to run at the same time?

 ksawerym.github.io/ComputerHardwareSimulators/#cc-slice

Solution: Simulator 4



Question 1

B: A, B, C, A, B, C, A, B.

Question 2

1000: one switch every 1 ms.

Question 3

No. Now you can see them take turns: the LED stops for a whole second while B and C run.

Concurrency works because the time slices are **short**: much shorter than anything we can notice.

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**



PC

where the task stopped: the next instruction

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**



PC

where the task stopped: the next instruction



Registers

the values it was working with, e.g. x5, x6 in Ripes

What does a task need to continue later?

If we stop task A now and resume it later, **what must we remember?**



PC

where the task stopped: the next instruction



Registers

the values it was working with, e.g. x5, x6 in Ripes



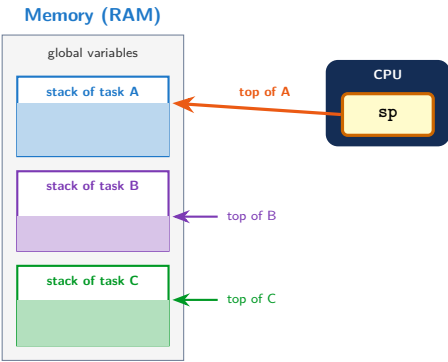
Stack

its local variables and the functions it is in

Together this is the task's **context**.

The ISR in Simulator 1 already saved t0: the same idea, but now we save **everything**.

Each task has its own stack

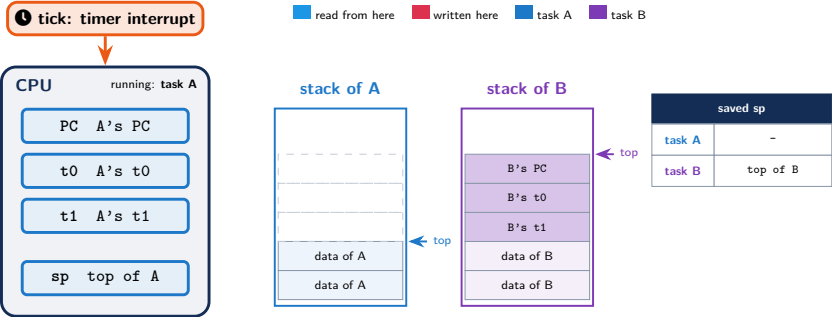


Its own stack
Each task has **its own stack** in memory: its local variables and saved registers.

The register sp
The stack pointer points to the stack of the task that is **running now**.

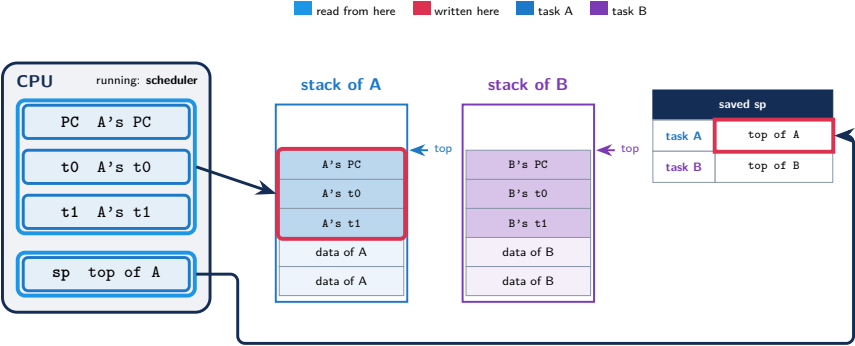
To switch tasks: save the registers on the old task's stack and **change sp**.

Context switch, step by step



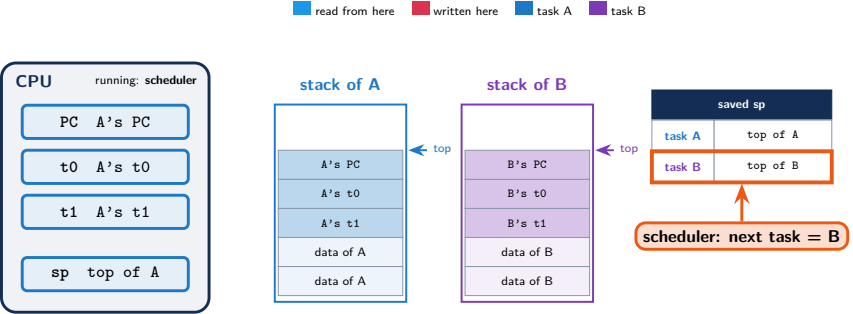
1 Tick
Task A runs. The timer interrupt arrives and the CPU jumps to the scheduler.

Context switch, step by step



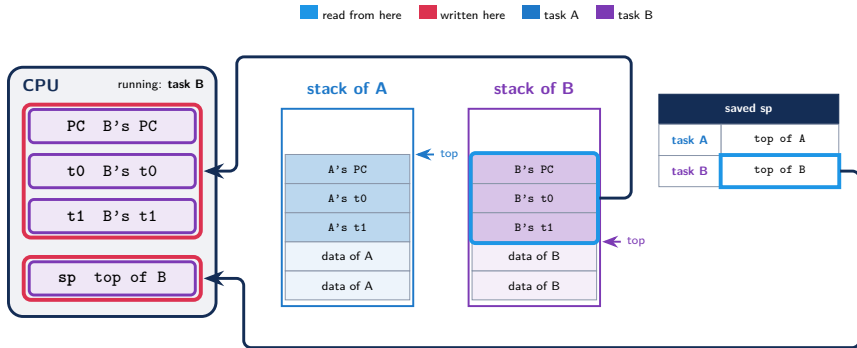
2 Save A
A's registers and PC go onto **stack A**. The new top of stack A goes into the table of saved stack pointers.

Context switch, step by step



3 Choose B
The scheduler picks the next task: **B**.

Context switch, step by step



4 Restore B

$sp \leftarrow$ the saved sp of B. B's registers and PC come back from stack B: B continues as if nothing happened. These 4 steps are a **context switch**.

Simulator 5: a context switch

Simulator task

Topic **Context switch**. Before each step, predict what will change. Then click **Next** and check.

At the start		
	CPU (task A runs)	saved on stack B
PC	0x0210	0x0308
t0	7	42
t1	3	9
sp	0x2000	saved sp of B: 0x2FF4

Each register takes **4 bytes**. A context = PC, t0, t1.

- Step 2, *Save A*: which values go onto stack A? What is the saved sp of A?
- Step 4, *Restore B*: what are PC, t0, t1 and sp after the restore?
- Where is A's t0 = 7 now? Who gives it back to A, and when?

 [ksawerym.github.io/ComputerHardwareSimulators/#cc-ctx](https://github.com/ksawerym/ComputerHardwareSimulators/#cc-ctx)

Solution: Simulator 5

Question 1

0x0210, 7 and 3: 3 registers $\times$ 4 bytes = 12 = 0xC.

Saved sp of A: 0x2000 – 0xC = 0x1FF4.

Question 2

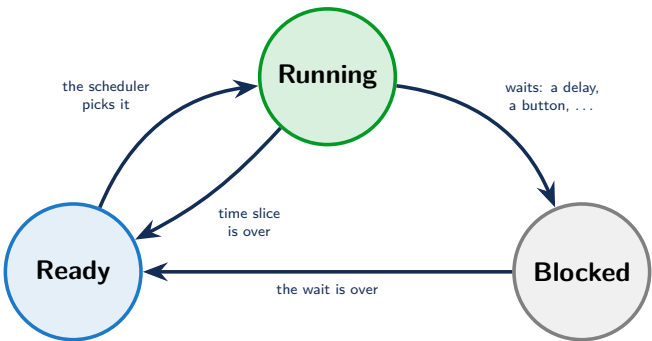
PC = 0x0308, t0 = 42, t1 = 9.
sp = 0x2FF4 + 0xC = 0x3000.

Question 3

On **stack A**. The scheduler restores it at the next switch back to A, exactly like B's values now.

Task A will never notice the break: its t0 waits for it on its own stack.

Task states



Running

On one core only **one** task runs at a time.

Ready

Could run, waits for its turn.

Blocked

Waits for something: a delay, a button. Gets **no** time slices.

Nobody is Ready?

Then the CPU runs a special task that does nothing: **IDLE**.

A Blocked task uses **no processor time**, unlike polling.

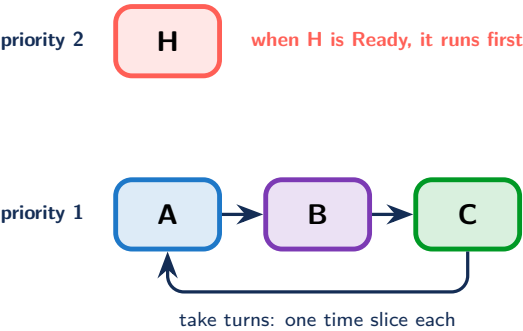
Which task runs next?

Round robin

Tasks with the **same priority** take turns, one time slice each.

Priorities

A Ready task with a **higher priority** always runs first. It even stops (preempts) a running task with a lower priority.



The scheduler picks the Ready task with the **highest priority**; equal priorities take turns.

Simulator 6: states, priorities and a shared counter

Simulator task

Topic **States and priorities**: A, B and C have priority 1. Task H has priority 2 and waits for the button.

1. At 2 ms, task C calls a 3 ms delay. In which state is C from 2 to 5 ms? Does it get any time slices?
2. Press the button: H becomes Ready. What happens to the task that is running?
3. What runs when A, B and C are all Blocked?

Then

Topic **Shared counter**: two tasks, each adds 1 to counter three times. The counter starts at 0.

4. **Predict** the final value.
5. Click **Run** three times and write down the results.

 [ksawerym.github.io/ComputerHardwareSimulators/#cc-states](https://github.com/Ksawerym/ComputerHardwareSimulators/#cc-states) [ksawerym.github.io/ComputerHardwareSimulators/#cc-shared](https://github.com/Ksawerym/ComputerHardwareSimulators/#cc-shared)

Solution: Simulator 6

Question 1

Blocked: no time slices. At 5 ms it becomes Ready again.

Question 2

It is stopped at once and goes to Ready. **H** runs. When H waits again, A, B and C continue.

Question 3

The **IDLE** task.

Questions 4 and 5

We expect **6**. But some runs give **4** or **5**, and the result changes from run to run.

Some additions were lost

Two tasks, six additions, and the counter did not count them all. Why? Keep this question: we will meet it again at the end of today.

A real scheduler: FreeRTOS

FreeRTOS is a small operating system for microcontrollers: mainly a **scheduler** plus tools that help tasks cooperate.

Free and open

Open source. Runs on many processors, including RISC-V.

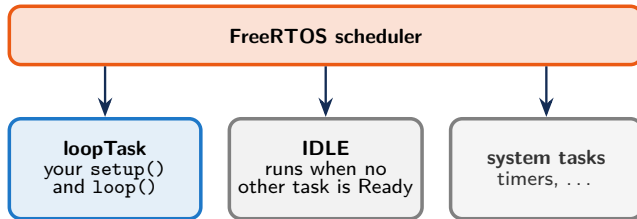
Real-time operating system: a task with a high priority reacts quickly.

Everywhere

Sensors, home appliances, cars, ...

Everything from Part 2 is inside: the tick, time slices, states, priorities and the context switch.

You have used FreeRTOS all along



On the ESP32, your `setup()` and `loop()` already run **inside a FreeRTOS task**.

On the ESP32, `delay()` makes the task **Blocked**: while your `loop()` waited, it did not waste the CPU.

Nothing to install: we can create our own tasks.

Task 3: who runs loop()?



```
void setup() {  
  Serial.begin(115200);  
  Serial.print("Task: ");  
  Serial.println(pcTaskGetName(NULL));  
  Serial.print("Priority: ");  
  Serial.println(uxTaskPriorityGet(NULL));  
}  
  
void loop() {  
  delay(1000);  
}
```

Task

Any Wokwi project: replace `setup()` and `loop()` with this code (also on [GitHub](#)). Run it and open the **Serial Monitor**.

1. Which task runs `setup()`?
2. What is its priority?

NULL means: the task that is running now.

Solution: who runs loop()?

Serial Monitor



Task: loopTask
Priority: 1

Questions 1 and 2

The task **loopTask**, with priority 1.

Next

The tasks we create will also get priority 1, so they take turns with loopTask.

Creating a task: xTaskCreate

```

void blinkTask(void *param) {
    for (;;) {                // forever
        digitalWrite(LED1, HIGH);
        vTaskDelay(pdMS_TO_TICKS(500));
        digitalWrite(LED1, LOW);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

void setup() {
    pinMode(LED1, OUTPUT);
    xTaskCreate(blinkTask, "blink", 2048, NULL, 1, NULL);
}

```

The arguments of xTaskCreate

blinkTask	the task function: an endless loop
"blink"	a name, for debugging
2048	stack size, on the ESP32 in bytes
NULL	a parameter for the task
1	the priority
NULL	a handle (not needed here)

vTaskDelay makes the task **Blocked**: it uses no processor time while it waits.

Task 4: three LEDs as three tasks

Task

The same three LEDs as in Task 2, but now **without the timer**. Template: [GitHub](#).

1. Remove the timer. Create **three tasks** with `xTaskCreate`: LED 1 changes every 300 ms, LED 2 every 500 ms, LED 3 every 700 ms.
2. Keep Loop: N on the LCD in `loop()`. Do the LEDs keep their rhythm?
3. In the task of LED 3, replace `vTaskDelay` with a **busy wait** (on the right). Do the other LEDs still blink? Now create this task with priority 2. What happens?

A busy wait for question 3



```
unsigned long t = millis();
while (millis() - t < 700) { }
```

Hint

One function can serve all three tasks: pass the pin and the period through `param`.

If you finish early: replace `vTaskDelay` with `vTaskDelayUntil`. What is the difference?

Solution: three LEDs as three tasks



```

struct Blink { int pin; int ms; };
Blink b1 = {4, 300}, b2 = {7, 500}, b3 = {3, 700};

void blinkTask(void *param) {
    Blink *b = (Blink *)param;
    pinMode(b->pin, OUTPUT);
    for (;;) {
        digitalWrite(b->pin, HIGH);
        vTaskDelay(pdMS_TO_TICKS(b->ms));
        digitalWrite(b->pin, LOW);
        vTaskDelay(pdMS_TO_TICKS(b->ms));
    }
}

// in setup():
xTaskCreate(blinkTask, "led1", 2048, &b1, 1, NULL);
xTaskCreate(blinkTask, "led2", 2048, &b2, 1, NULL);
xTaskCreate(blinkTask, "led3", 2048, &b3, 1, NULL);

```

Question 2

Yes. Each LED task is Blocked most of the time and gets the core within 1 ms when its delay is over.

Question 3, priority 1

The other LEDs still blink, but the busy task wastes its time slices: **polling again**.

Question 3, priority 2

The other LEDs and the LCD **stop**: the busy task is always Ready and has the highest priority.

If you finished early

vTaskDelayUntil wakes the task at fixed moments: small delays do not add up.

Two solutions of the same problem

	Task 2: one timer + ticks	Task 4: three FreeRTOS tasks
Who counts the ticks	you, in the ISR	the scheduler
The code of one LED	mixed into one ISR	its own simple loop
A fourth LED	change the ISR	one more xTaskCreate
Waiting	-	vTaskDelay: Blocked, no CPU
Cost	nothing extra	one stack per task (RAM)

Both use a timer tick. With FreeRTOS, the scheduler does the bookkeeping for you. Next: **look inside**, what did it do?

Look inside: a task in FreeRTOS

tasks.c (simplified)

```

    ● ● ●
typedef struct tskTaskControlBlock {
    volatile StackType_t *pxTopOfStack; /* MUST be first */
    ListItem_t      xStateListItem; /* Ready or Blocked list */
    UBaseType_t     uxPriority;      /* priority */
    StackType_t     *pxStack;      /* start of the stack */
    char            pcTaskName[16]; /* name */
    /* ... */
} tskTCB;
  
```

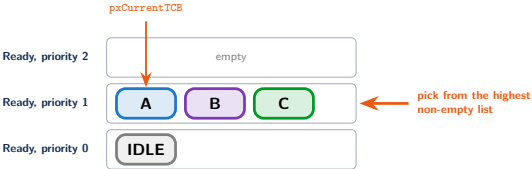
- pxTopOfStack**
 The **saved sp** from our context switch. It is the **first** field.
- xStateListItem**
 On which list the task is: **Ready** or **Blocked**.
- uxPriority**
 Its priority: 1 for our tasks.

TCB = Task Control Block: one per task. It is our table of saved stack pointers, and more.

Choosing the next task: vTaskSwitchContext

tasks.c (simplified)

```
void vTaskSwitchContext( void )
{
    /* ... */
    taskSELECT_HIGHEST_PRIORITY_TASK();
    /* now pxCurrentTCB points to
       the TCB of the next task */
}
```



Step 3 of the context switch

The scheduler takes the first task from the **highest non-empty** Ready list and stores its TCB in **pxCurrentTCB**. Tasks with the same priority take turns.

Saving and restoring registers (simplified)

Save the old task (step 2)

```

    ● ● ●
    addi sp, sp, -CONTEXT_SIZE # room on its stack
    sw   ra, 1*4(sp)           # save registers
    sw   t0, 2*4(sp)
    ...
    csrr t0, mepc              # where it stopped
    sw   t0, 0(sp)             # save the PC
    lw   t0, pxCurrentTCB      # its TCB
    sw   sp, 0(t0)             # pxTopOfStack = sp
  
```

Restore the new task (step 4)

```

    ● ● ●
    lw   t0, pxCurrentTCB      # the new TCB
    lw   sp, 0(t0)             # sp = pxTopOfStack
    lw   t0, 0(sp)             # its saved PC
    csrw mepc, t0
    lw   ra, 1*4(sp)           # restore registers
    lw   t0, 2*4(sp)
    ...
    addi sp, sp, CONTEXT_SIZE
    mret                        # continue the task
  
```

Exactly our 4 steps from Part 2, in real code: lw and sw as in Ripes, and one TCB per task.

Simulator 7: the context switch with FreeRTOS names

Simulator task

Topic **Context switch** again, the same start as in Simulator 5. Turn on *FreeRTOS names*. **Before each step**, predict, then click **Next**.

1. Before step 2: to which TCB does `pxCurrentTCB` point?
2. After step 2: what is the value of `pxTopOfStack` in the TCB of A?
3. Step 3: what does `vTaskSwitchContext` change?
4. Step 4: which line of the assembly gives `sp` its new value? What is that value?

 [ksawerym.github.io/ComputerHardwareSimulators/#cc-ctx](https://github.com/Ksawerym/ComputerHardwareSimulators/#cc-ctx)

Solution: Simulator 7

Question 1

To the TCB of **A**: A was running.

Question 2

0x1FF4: the same saved sp as in Simulator 5.

Question 3

Only pxCurrentTCB: now it points to the TCB of **B**.

Question 4

lw sp, 0(t0), with t0 = the TCB of B: sp = 0x2FF4.
After the registers are restored, sp = 0x3000.

The table of saved stack pointers from Part 2 is the field pxTopOfStack in every TCB.

Task 5: a race between two tasks



```
const int N = 1000000;
volatile int counter = 0;
volatile bool doneA = false, doneB = false;

void incTask(void *param) {
    for (int i = 0; i < N; i++) {
        counter++;
    }
    *(volatile bool *)param = true;    // done
    vTaskDelete(NULL);                 // end this task
}
```

Task

Open the template from [GitHub](#). Two tasks increase one shared counter.

1. Start **two** incTask tasks with the same priority, with &doneA and &doneB as parameters.
2. In loop(), wait until both are done. Then show on the LCD: Goal: 2000000 and Got: N.
3. Run it several times. Is N always the same? Is it ever correct?

If you finish early: set N = 1000. Is the result correct now? Why?

Solution: a race between two tasks



```

void setup() {
    // the LCD as before
    xTaskCreate(incTask, "incA", 2048,
                (void *)&doneA, 1, NULL);
    xTaskCreate(incTask, "incB", 2048,
                (void *)&doneB, 1, NULL);
}

void loop() {
    if (doneA && doneB) {
        lcd.setCursor(0, 0);
        lcd.print("Goal: ");  lcd.print(2 * N);
        lcd.setCursor(0, 1);
        lcd.print("Got: ");   lcd.print(counter);
        while (true) delay(1000);
    }
    delay(10);           // wait without wasting the CPU
}
    
```

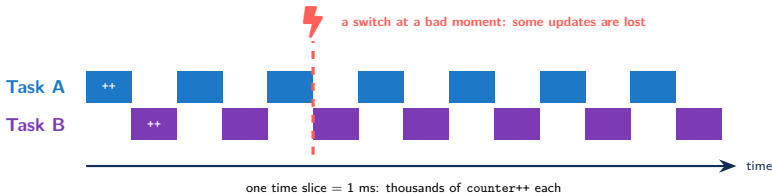
Question 3

N is **smaller** than 2 000 000 and **different** every time. It is never correct.

If you finished early

With N = 1000 each task finishes within one time slice: no switch comes in the middle, so the result is correct.

What happened?



This is a **race condition**: the result depends on **when** the scheduler switches the tasks.

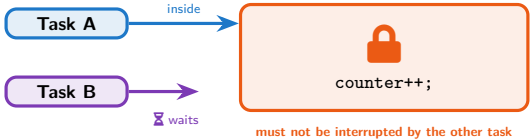
We saw it three times
Two cores: two presses, presses grew by one. Simulator 6: 6 additions, counter 4 or 5. Task 5: two million additions, the counter got less.

The question

Why? `counter++` is **one line**. How can a switch fall *in the middle* of it?

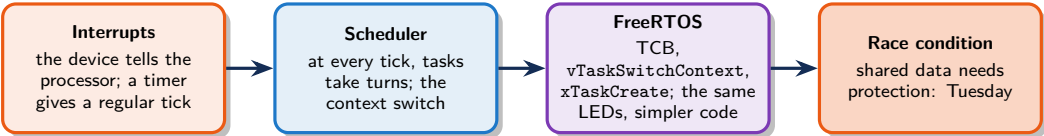
How do we fix this?

We need a way to say: *while I update the counter, nobody else may touch it.*



Tomorrow: **why** updates are lost, and how to fix it with **threads, mutexes and atomic operations**, in C++ on your laptops.

What we saw today



A brief history of concurrency: it all started with a button.

Check yourself

1. Why does an interrupt not miss a short press, while polling can?
2. What does the CPU save when an interrupt arrives, and why does the main program not notice the break?
3. What is a context switch? What is saved, and where?
4. Why does a task that waits with `vTaskDelay` not waste CPU time, while a busy loop does?

Try to answer each question in **2 or 3 sentences**.

Before Tuesday

From Tuesday we write **C++** on your own laptops. Install a compiler with **C++20** support (g++, clang or MSVC). Instructions will be sent today.

hello.cpp: check your installation

```
● ● ●
#include <iostream>
#include <thread>

int main() {
    std::thread t([] {
        std::cout << "Hello from a thread\n";
    });
    t.join();
}
```

Compile and run

```
● ● ●
g++ -std=c++20 -pthread hello.cpp -o hello
./hello
```

If it prints Hello from a thread, you are ready for Tuesday.